

Classical Baseline: CG Arithmetic, HPCG Runtime and Energy

HHL paper revision — restarted Task 08

20 September 2026

Abstract

This report implements the revised September 20 task: retain the manuscript’s CG-family arithmetic expression, convert its FLOP count to runtime using published HPCG PFLOP/s, and multiply runtime by system power to estimate energy. The result is a transparent, executable solver-core projection. Four pinned hardware scenarios use the November 2025 HPCG scores and same-edition TOP500 power proxies. The source expression, integer-iteration convention, units, rate-transfer assumptions and power provenance are explicit. No direct solver or independent multigrid cost model is included in the active comparison.

Main equations. For the paper’s parameters (N, κ, s, ϵ) and a hardware score R_H in PFLOP/s,

$$n_{\text{FLOPs}} = \left\lceil \frac{\kappa}{2} \ln \frac{2}{\epsilon} \right\rceil (4Ns + 14N), \quad (1)$$

$$t_{\text{core}} = \frac{n_{\text{FLOPs}}}{10^{15} R_H} \quad \text{seconds}, \quad (2)$$

$$E_{\text{core}} = 10^3 P_{\text{kW}} t_{\text{core}} \quad \text{joules}. \quad (3)$$

The literal, unrounded Eq. (D1) is also exported. The equations are arithmetic and benchmark-transfer models, not exact executed-instruction counts or measured application runtime/energy.

Scope change. Superseded LU, Cholesky, multigrid and stress-test reports have been removed. The active deliverable is this CG/HPCG report. The new model requires only Python’s standard library and does not run a production solver or HPCG.

Two essential provenance distinctions. The paper’s Figure 13 recurrence is a normal-equation CG variant, despite inconsistent naming in the manuscript. Also, TOP500 power is not verified paired HPCG-run power. This report retains the requested formula and conversion without obscuring either limitation.

1 Logical arithmetic extracted from the manuscript

The source is Appendix D, p. 24, Figure 13, Theorem 4 and Eq. (D1). Appendix F, p. 28, Eq. (F1) already expresses energy as work divided by a rate, multiplied by power. This revision changes the physical rate to HPCG FLOP/s.

Parameter	Meaning
N	Original square matrix dimension
s	Maximum nonzeros per row and column
κ	Original spectral condition number
ϵ	Paper's relative solution-error parameter, $0 < \epsilon < 1$
K	Iteration count, distinct from κ
R_H, P_{kW}	HPCG PFLOP/s and system power in kW

The manuscript decomposes an iteration as follows:

Work	Modeled FLOPs
Two sparse matrix-vector products	$4Ns$
Three vector updates	$6N$
Coefficient calculations	$8N$
Total	$4Ns + 14N$

The paper states an iteration threshold $K \geq (\kappa/2) \ln(2/\epsilon)$, giving the literal expression

$$\boxed{f_{D1}(N, \kappa, s, \epsilon) = \frac{\kappa}{2} \ln \frac{2}{\epsilon} (4Ns + 14N).} \quad (4)$$

For a whole-iteration workload, the executable defaults to

$$K_{\text{model}} = \left\lceil \frac{\kappa}{2} \ln \frac{2}{\epsilon} \right\rceil, \quad n_{\text{FLOPs}} = K_{\text{model}}(4Ns + 14N). \quad (5)$$

Both versions are recorded. Their difference is nonnegative and less than one modeled iteration. Natural logarithms are used. The stated scaling is $O(Ns\kappa \ln(1/\epsilon))$.

1.1 What the expression does and does not count

Equation (4) is the exact transcription of a *paper arithmetic expression*. It is not an exact ISA instruction count: Ns substitutes for actual nonzeros, coefficient calculations are grouped into $8N$, and initialization, addressing, integer work, input construction, output and certification are omitted. An FMA may be one instruction but counts as two FLOPs. The paper does not specify a precision-dependent instruction implementation; the HPCG transfer assumes real FP64 arithmetic. The iteration formula is inherited as a model, not newly established as a finite-precision convergence certificate.

1.2 CG versus the paper's normal-equation variant

Figure 13 calls its algorithm CGNE, but updates $z = A^T r$, $p = z + \beta p$, and x directly. This is CGLS/CGNR, implicit CG on $A^T A x = A^T b$. The normal operator squares conditioning, explaining the two operator actions and linear original- κ dependence. Ordinary SPD CG has

a different iteration cost and conditioning dependence [1]. We therefore retain the label *paper CG-family baseline (D1)* and record the variant, rather than silently claim Eq. (4) is ordinary SPD CG. No second solver is added. Theorem 4’s error statement is transcribed, not re-proved here.

2 HPCG physical computing model

Let R_H be the published whole-configuration HPCG score. The requested conversion is

$$t_{\text{core}}[s] = n_{\text{FLOPs}} / (10^{15} R_H), \quad (6)$$

$$E_{\text{core}}[J] = 10^3 P_{\text{kW}} t_{\text{core}}, \quad (7)$$

$$E_{\text{core}}[kWh] = E_{\text{core}}[J] / (3.6 \times 10^6). \quad (8)$$

The score already aggregates benchmark performance across its configuration. Do not multiply again by cores, the displayed fraction of peak, or a generic parallel-efficiency factor. Use the HPCG column, not the neighboring HPL Rmax column.

2.1 Pinned hardware scenarios

We pin the November 2025 lists, retrieved 20 September 2026. This is an explicit edition choice, not a claim to use the newest possible ranking. HPCG scores come from the official HPCG list [2]; power comes from the same-edition TOP500 page [3]. Published names and listed core counts match, but this does not establish identical measurement runs.

System	HPCG (PF/s)	Power (MW)	Time (s)	Energy (kWh)
El Capitan	17.410	29.685	57.438	473.626
Fugaku	16.000	29.899	62.500	519.080
Frontier	14.050	24.607	71.174	486.497
Aurora	5.613	38.698	178.158	1915.098

Table 1: Published rate and power inputs, followed by derived time and energy for 10^{18} modeled FLOPs. This unit-work example has no implied matrix size or capacity certification. Power is a proxy, not verified contemporaneous HPCG power.

The source rates are 17.41, 16.00, 14.05 and 5.613 PFLOP/s. Powers are 29,685, 29,899, 24,607 and 38,698 kW, respectively. These replace the original 1 GHz/50 W desktop model. Source snapshots and hashes accompany the frozen JSON values. No hardware measurement was performed in this task.

The coefficient P/R_H combines two source quantities; it is not measured HPCG energy efficiency. Metering scope, facility/cooling inclusion and gross versus incremental allocation are not determined by the selected power table. No PUE multiplier is inferred. Task 07 must align the energy boundary with the quantum branch before publication of an energy ratio.

2.2 Why this is a transfer assumption

HPCG combines sparse matvecs, vector updates, reductions and smoothing in a multigrid-preconditioned CG workload [4]. That is not the paper’s normal-equation recurrence. Applying the measured HPCG rate to D1 is the user’s requested physical proxy, not evidence that D1’s workload sustains that rate. Benchmark memory and synchronization effects are already reflected in the score; application differences remain unresolved.

HPCG’s FAQ requires FP64 computations and large official runs, normally at least 1800 seconds with substantial main-memory use [5]. A small matrix cannot simply saturate a whole-supercomputer score. Very short results from the conversion are formal core-time projections, not observed latency. We do not add an independent multigrid, bandwidth or communication model in this narrowed revision.

For sensitivity, define an application-to-HPCG rate multiplier $\eta > 0$ and power multiplier $\rho > 0$:

$$t(\eta) = \frac{n_{\text{FLOPs}}}{\eta 10^{15} R_H}, \quad E(\eta, \rho) = \rho 10^3 P_{\text{kW}} t(\eta). \quad (9)$$

The nominal case is $\eta = \rho = 1$. The supplied grid uses $\eta \in \{0.25, 0.5, 1, 2\}$ and $\rho \in \{0.7, 1, 1.3\}$, giving 48 rows over four systems. These are scenario multipliers with no probabilistic meaning; η is not a second benchmark parallel-efficiency correction.

3 Application, memory and completeness

Task 02 now provides `02-smooth-diffusion-v1`, with default $m = 127$, $N = 16,129$, $\beta = 9$, $s = 5$ and continuum output tolerance $\epsilon_{\text{out}} = 10^{-4}$. The functional is $c^\top x$ with $c = h^2 \mathbf{1}$. Its manufactured continuum answer is $J = 5/144$, directly available to an unrestricted classical algorithm. Any solver comparison must state this known-answer limitation.

D1’s ϵ is not ϵ_{out} . Conditional on the paper’s relative solution-error claim and $\lambda_{\min}(A) \geq \lambda_0$,

$$|c^\top(\hat{x} - x)| \leq \|c\|_2 \epsilon \|x\|_2 \leq \frac{\|c\|_2 \|b\|_2}{\lambda_0} \epsilon. \quad (10)$$

Task 02 gives $\|c\|_2 = h^2 \sqrt{N}$, $\|b\|_2 \leq \sqrt{N} F$, $F = 3(1 + \beta) + 9\beta/16$, and an algebraic allowance $\epsilon_{\text{alg}} = \epsilon_{\text{out}}/4$. A sufficient conditional adapter is

$$\epsilon_{D1} = \frac{\epsilon_{\text{alg}} \lambda_0}{h^2 N F}, \quad \lambda_0 = 8h^{-2} \sin^2(\pi h/2), \quad (11)$$

$$\kappa_{\text{scenario}} = (1 + \beta) \cot^2(\pi h/2). \quad (12)$$

Original κ remains unknown; the last expression is its upper-bound scenario. The supplied adapter JSON evaluates these quantities and checks mesh admissibility. It is not a new solve or an accuracy/reliability certificate. Representation, arithmetic, discretization and delivery-failure budgets remain separate.

The common input starts as a compact analytic specification. Generation, initialization, transfers, output and final residual checks are not paid by D1. Thus the executable labels its outputs *solver core only*, leaves end-to-end fields null, and never marks a row eligible for an advantage claim. It does not silently assign zero to missing phases.

At least one explicit FP64 vector requires $8N$ bytes. A supplied memory limit can reject rows that fail even this lower bound; passing does not prove total solver-memory feasibility. At $N = 2^{100}$ one vector needs approximately 1.014×10^{31} bytes. The earlier direct-solver crossover and 200-hour claim are outside the revised scope and are not carried forward. Changing the denominator from CPU frequency to HPCG alone cannot validate a crossover.

4 Reproducibility and manuscript handoff

Run from the repository root with Python 3.6 or newer; no third-party package is required for the arithmetic model:

```
python3 paper-reverse/results/08-classical-baseline/reproduce.py
python3 paper-reverse/results/08-classical-baseline/cg_hpcg.py
```

The default input is $N = 2^{30}$, $\kappa = 100$, $s = 5$, $\epsilon = 10^{-3}$, with integer-iteration rounding. It is a generic scaling scenario, not a measured task-02 instance. Pass `--rounding paper` for literal D1. Optional rate, power and memory-limit arguments are documented in the package README.

Validation checks a hand-computable D1 case, integer rounding, unit conversions, rate/power scaling, a memory lower-bound rejection, invalid inputs and the task-02 mesh/accuracy adapter. Outputs comprise the default model JSON, 16 scaling rows, 48 sensitivity rows, source/input hashes and a validation log. These validate model arithmetic and bookkeeping, not actual application performance or the inherited theorem. No HPCG or production solver execution is claimed.

The task-01 mapping is $pp = (N, s, \kappa, \text{output/access contract})$, with unknown original conditioning preserved as an interval/scenario where appropriate. The selected *cc* is the paper's normal-equation CG variant, FP64 rate-transfer assumption, pinned HPCG profile and model version `08-cg-paper-hpcg-v1`. Arithmetic FLOPs map to classical operations; the core-only hardware result cannot be labeled a full-workload evaluated record. Task 05 owns missing input/output and reliability accounting; task 07 owns common energy boundaries.

Replacement main-text and appendix passages are delivered in `manuscript-text.md`. They state D1, the HPCG time/energy equations, the variant distinction and the proxy nature of power. C1-4 is addressed through published benchmark rates; C1-3's naming problem is made explicit, but retaining D1 does not satisfy the reviewer's request for an ordinary-SPD/competitive-preconditioned comparison. C2-3 is supported by a scientific-computing proxy and the application adapter, not by new production measurements. Task 09 must resolve accuracy, application-rate transfer, capacity and complete energy scope before deriving advantage ratios.

All shared code and the manuscript PDF are preserved. The September 20 task supersedes the previous direct-solver and independent multigrid-model requirements; the superseded reports and archive have been removed at the user's request.

References

- [1] R. Barrett et al. *Templates for the Solution of Linear Systems*. Netlib/SIAM. <https://www.netlib.org/templates/templates.pdf>.
- [2] HPCG. *November 2025 HPCG Results*. <https://www.hpcg-benchmark.org/custom/sc25.html>.
- [3] TOP500. *November 2025 list and power table*. <https://www.top500.org/lists/top500/2025/11/>.
- [4] HPCG project. *Benchmark description*. <https://www.hpcg-benchmark.org/>.
- [5] HPCG. *FAQ*. <https://www.hpcg-benchmark.org/faq/index.html>.

Hardware sources retrieved 20 September 2026; the source edition is pinned to November 2025. The local manuscript supplies D1 and F1; task-01/02 inputs and their versions are recorded in the manifest.